Agent Swarm - Stack Integrado (RTK + Context Mode + Engram + Covacha MCP)

Fecha: 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Arquitectura definitiva **Principio:** Cada herramienta resuelve UNA capa. Cero duplicacion.

---

1. El Stack de 4 Capas

Cada capa resuelve un problema distinto. Ninguna se traslapa.

```
┌──────────────────────────────────────────────────────────────┐
│                    CLAUDE CODE SESSION                        │
│                                                               │
│  CAPA 4 ── Covacha MCP Server (TypeScript, DynamoDB)          │
│  Coordinacion: locks, deps, msgs, contratos, roles, eval      │
│  → Quién hace qué, cuándo, y cómo se comunican               │
│                                                               │
│  CAPA 3 ── Engram (Go binary, SQLite + FTS5)                  │
│  Memoria: decisiones, patrones, bugs, arquitectura            │
│  → Qué aprendí y qué recuerdo entre sesiones                 │
│                                                               │
│  CAPA 2 ── Context Mode (Node.js, SQLite + FTS5)              │
│  Sesion: sandbox, session continuity, compaction survival     │
│  → Qué estoy haciendo AHORA y no perder el hilo              │
│                                                               │
│  CAPA 1 ── RTK (Rust binary, hooks)                           │
│  Tokens: filtrado, compresion, deduplicacion de CLI output    │
│  → Gastar menos tokens en output de comandos                  │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

Responsabilidades claras (SIN overlap)

Capa	Herramienta	Resuelve	Storage	Scope
1. Tokens	RTK	Output de CLI es muy largo, quema tokens	SQLite local (history.db)	Por comando
2. Sesion	Context Mode	Al compactar pierde archivos/tareas/decisiones en progreso	SQLite local (per-project)	Por sesión
3. Memoria	Engram	Entre sesiones olvida decisiones, patrones, bugs resueltos	SQLite local (engram.db)	Cross-sesión
4. Coordinacion	Covacha MCP	N máquinas necesitan coordinar trabajo sin colisiones	DynamoDB remoto	Cross-máquina

Flujo de datos

```
git status
    |
    ▼
RTK (Capa 1): 2000 tokens → 200 tokens (filtrado)
    |
    ▼
Context Mode (Capa 2): Captura evento "git status en branch X"
    |                   Si output > 5KB → sandbox + intent search
    |
    ▼
Claude Code procesa (200 tokens en vez de 2000)
    |
    ▼
Si es decision importante:
    |
    ▼
Engram (Capa 3): mem_save("Decidimos usar branch por feature, no trunk-
    |
    ▼
Si completa tarea:
    |
    ▼
Covacha MCP (Capa 4): covacha_release_task → MSG → resolve_deps → publi
```

---

2. Interaccion Entre Capas

2.1 RTK + Context Mode (no compiten)

RTK filtra output de Bash ANTES de que entre al contexto. **Context Mode** procesa output en sandbox DESPUES de Bash.

```
Sin RTK + Sin CM:  git log → 2000 tokens en contexto
Con RTK + Sin CM:  git log → rtk git log → 200 tokens en contexto
Sin RTK + Con CM:  git log → ctx_execute("git log") → 107 bytes en cont
Con RTK + Con CM:  git log → rtk intercepta → 200 tokens → CM captura e
```

RTK actua en Capa 1 (hook pre-tool, reescribe comandos Bash). **Context Mode actua en Capa 2** (MCP server, sandbox para outputs grandes).

Regla de coexistencia:

- RTK maneja comandos simples (git, ls, grep, test runners)
- Context Mode maneja outputs masivos (Playwright snapshots, JSON APIs, web scraping)
- No hay conflicto: RTK es un hook de Bash, CM es un MCP server

2.2 Context Mode + Engram (no compiten)

Context Mode recuerda qué pasa DENTRO de una sesión (archivos editados, tareas, errores). **Engram** recuerda qué aprendiste ENTRE sesiones (decisiones, patrones, arquitectura).

```
Session 1:
```

```
  Context Mode: "Editaste auth.py, user.py, hay 2 tareas pendientes"
  Engram: mem_save("JWT auth usa RS256, no HS256. Razon: multi-tenant r

Session 2 (nueva):
  Context Mode: (vacio — session nueva)
  Engram: mem_search("auth") → "JWT usa RS256 por multi-tenant"
  → El agente recuerda decisiones pasadas aunque sea session nueva
```

Regla de coexistencia:

- Context Mode maneja eventos de sesión (PostToolUse, PreCompact, SessionStart)
- Engram maneja conocimiento persistente (mem_save, mem_search, mem_context)
- No hay conflicto: diferentes SQLite DBs, diferentes hooks

2.3 Engram + Covacha MCP (complementarios)

Engram guarda conocimiento LOCAL del agente (1 nodo). **Covacha MCP** comparte contexto entre N nodos.

```
Nodo-1 termina ISS-043:
  Engram: mem_save("SPEI endpoint usa Decimal para montos, no float. Bu
  Covacha: covacha_release_task(043, done)
           covacha_publish_contract("SPEI Transfer v1", POST /api/v1/sp
           covacha_send_message("task_completed", task=043)

Nodo-2 empieza ISS-044 (frontend para SPEI):
  Covacha: covacha_claim_task(044) → lee CONTRACT#spei-transfer-v1
  Engram: mem_search("SPEI") → (vacio, es OTRO nodo, no comparte SQLite
  → Pero el contrato de Covacha le da el schema que necesita
```

Regla de migración de learnings: Cuando Covacha MCP release_task con learnings, los guarda en DynamoDB LEARNING# (compartido). Engram guarda en SQLite local (personal del nodo).

```
# Al release, guardamos en AMBOS:
# 1. Engram (memoria personal del nodo)
engram.mem_save(
    title="SPEI usa Decimal no float",
    type="bugfix",
    content="**What**: Montos SPEI deben ser Decimal\n**Why**: Float ro
)

# 2. Covacha (learning compartido con otros nodos)
covacha.save_learning(
    module="covacha-payment",
    learning="Usar Decimal para montos SPEI, float causa rounding error
)
```

2.4 RTK + Covacha MCP (independientes)

RTK no sabe que Covacha existe y viceversa. RTK comprime output de comandos. Covacha coordina tareas entre nodos. No interactúan.

---

3. Setup por Nodo

3.1 Instalacion completa (las 4 capas)

```
# ─── CAPA 1: RTK (ya instalado en BAAT) ───
brew install rtk
rtk init --global
# Verificar: rtk --version && rtk gain

# ─── CAPA 2: Context Mode ───
# En Claude Code:
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode
# Verificar: /context-mode:ctx-doctor

# ─── CAPA 3: Engram ───
brew install gentleman-programming/tap/engram
# En Claude Code:
claude plugin marketplace add Gentleman-Programming/engram
claude plugin install engram
# Verificar: engram version && engram stats

# ─── CAPA 4: Covacha MCP ───
npm install -g covacha-mcp-server  # (cuando esté publicado)
covacha init --tenant baatdigital --org baatdigital
# Verificar: covacha status
```

3.2 Configuracion Claude Code (.claude/settings.json)

```
{
  "mcpServers": {
    "context-mode": {
      "command": "context-mode"
    },
    "engram": {
      "command": "engram",
      "args": ["mcp"]
    },
    "covacha": {
      "command": "covacha-mcp-server"
    }
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "command": "rtk hook pretooluse"
      },
      {
        "matcher": "Bash|Read|Grep|WebFetch",
        "command": "context-mode hook claude-code pretooluse"
      }
```

```
    ],
    "PostToolUse": [
      {
        "command": "context-mode hook claude-code posttooluse"
      }
    ],
    "PreCompact": [
      {
        "command": "context-mode hook claude-code precompact"
      }
    ],
    "SessionStart": [
      {
        "command": "context-mode hook claude-code sessionstart"
      }
    ]
  }
}
```

3.3 Herramientas MCP disponibles (31 tools)

```
RTK (0 MCP tools — es hook de Bash, no MCP server):
  → Transparente, reescribe comandos automaticamente

Context Mode (6 MCP tools):
  ctx_execute          → Ejecutar codigo en sandbox
  ctx_batch_execute    → Multiples comandos en 1 llamada
  ctx_execute_file     → Procesar archivo en sandbox
  ctx_index            → Chunk markdown en FTS5
  ctx_search           → Query sobre contenido indexado
  ctx_fetch_and_index  → Fetch URL + chunk + index

Engram (13 MCP tools):
  mem_save             → Guardar observacion (decision/bug/pattern)
  mem_update           → Actualizar por ID
  mem_delete           → Borrar (soft/hard)
  mem_suggest_topic_key→ Sugerir topic_key estable
  mem_search           → Busqueda full-text
  mem_session_summary  → Resumen de fin de sesion
  mem_context          → Contexto de sesiones anteriores
  mem_timeline         → Contexto cronologico
  mem_get_observation  → Contenido completo por ID
  mem_save_prompt      → Guardar prompt del usuario
  mem_stats            → Estadisticas
  mem_session_start    → Registrar inicio de sesion
  mem_session_end      → Marcar sesion como completada

Covacha MCP (14 MCP tools):
  covacha_bootstrap        → Generar contexto (tareas, learnings, equip
  covacha_claim_task       → Reclamar tarea con lock atomico
  covacha_release_task     → Liberar tarea + learnings + contract
  covacha_send_message     → Enviar mensaje a otros nodos
  covacha_read_messages    → Leer mensajes pendientes
```

```
covacha_check_deps       → Verificar/agregar dependencias
covacha_publish_contract → Publicar contrato de API
covacha_get_contract     → Leer contrato
covacha_evaluate_ticket  → Auto-evaluacion (pre_release/qa/po)
covacha_role_action      → Accion segun rol (tech_lead/tester/po)
covacha_swarm_status     → Estado del swarm
covacha_workspace_list   → Workspaces del tenant
covacha_workspace_switch → Cambiar workspace activo
covacha_heartbeat        → Ping de vida
```

---

4. Lifecycle Integrado de una Tarea

4.1 Sesion completa de un nodo

```
┌──────────────────────────────────────────────────────────────────┐
│ INICIO DE SESION                                                 │
│                                                                  │
│ 1. RTK hook se activa automaticamente                            │
│ 2. Context Mode SessionStart: restaura estado si --continue      │
│ 3. Engram: mem_session_start + mem_context (sesiones anteriores) │
│ 4. Covacha: covacha_bootstrap (tareas, learnings, equipo)        │
│    → El nodo sabe: que hacer, que aprendio, quien mas trabaja    │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│ CLAIM TAREA                                                      │
│                                                                  │
│ 5. Covacha: covacha_claim_task(ISS-043)                          │
│    → Lock atomico en DynamoDB                                    │
│    → GitHub Board → "In Progress"                                │
│    → Carga contratos relevantes                                  │
│ 6. Engram: mem_search("SPEI") → decisiones pasadas del modulo    │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│ IMPLEMENTACION                                                   │
│                                                                  │
│ 7. Cada git/test/build pasa por RTK (80% menos tokens)           │
│ 8. Context Mode captura: archivos editados, errores, git ops     │
│ 9. Si se compacta: Context Mode restaura estado automaticamente  │
│ 10. Engram: mem_save al tomar decisiones de arquitectura         │
│ 11. Covacha: covacha_heartbeat cada 2 min                        │
│ 12. Covacha: covacha_read_messages (mensajes de otros nodos)     │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│ PRE-RELEASE (auto-evaluacion)                                    │
│                                                                  │
│ 13. Covacha: covacha_evaluate_ticket(043, "pre_release")         │
│     → Tests pasan? Coverage >= 98%? Lint limpio?                 │
│     → Type hints? Commit format? No secrets?                     │
```

```
│     Si FAIL: fix → re-evaluate (max 3 intentos)                  │
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────┐
│ RELEASE                                                          │
│                                                                  │
│ 14. Covacha: covacha_release_task(043, done, learnings=[...])    │
│     → Guarda learnings en DynamoDB (compartido)                  │
│     → Publica contrato si es backend endpoint                    │
│     → Resuelve dependencias (desbloquea ISS-044)                 │
│     → MSG broadcast: "ISS-043 done"                              │
│     → GitHub: cierra issue, board → "Done"                       │
│                                                                  │
│ 15. Engram: mem_save(learnings como observaciones persistentes)  │
│     → Topic key para que evolucione si se descubre mas           │
│                                                                  │
│ 16. Engram: mem_session_summary("Implementé endpoint SPEI...")   │
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────┐
│ QA + ACEPTACION (otros nodos)                                    │
│                                                                  │
│ 17. Nodo-Tester recibe MSG "ISS-043 done"                        │
│     → Covacha: covacha_evaluate_ticket(043, "qa_review")         │
│     → Si PASS: MSG "qa_approved"                                 │
│     → Si FAIL: MSG "qa_rejected" + detalles                      │
│                                                                  │
│ 18. Nodo-PO recibe MSG "qa_approved"                             │
│     → Covacha: covacha_evaluate_ticket(043, "po_acceptance")     │
│     → Si PASS: acepta ticket, actualiza epica                    │
│     → Si FAIL: MSG "po_rejected" + criterios faltantes           │
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────┐
│ SIGUIENTE TAREA                                                  │
│                                                                  │
│ 19. Covacha: find_next_task (respeta deps, caps, scoring)        │
│ 20. Si no hay tareas en este workspace:                          │
│     → auto_switch_workspace → busca en otro workspace            │
│ 21. GOTO paso 5 (claim)                                          │
└──────────────────────────────────────────────────────────────────┘
```

---

5. Engram Memory Protocol para el Swarm

5.1 Que guardar en Engram (per-nodo)

```
SIEMPRE guardar:
  - Decisiones de arquitectura (topic_key: "architecture/*")
  - Bugs encontrados y su solucion (topic_key: "bug/*")
  - Patrones descubiertos (topic_key: "pattern/*")
  - Configuraciones no obvias (topic_key: "config/*")
```

```
  - Session summaries al terminar cada tarea

NUNCA guardar:
  - Output de comandos (RTK se encarga)
  - Estado de sesion actual (Context Mode se encarga)
  - Tareas/locks/mensajes (Covacha se encarga)
  - Datos que estan en git (el repo es la fuente de verdad)
```

5.2 Formato estandarizado de observaciones

```
mem_save:
  title: "SPEI endpoint usa Decimal para montos"
  type: "bugfix"
  topic_key: "bug/spei-decimal-montos"
  content: |
    **What**: Montos en transferencias SPEI deben usar Decimal, no floa
    **Why**: Float rounding causa diferencia de centavos (0.01) en conc
    **Where**: covacha-payment/services/spei_service.py:45
    **Learned**: Regla global — TODOS los montos en el ecosistema deben
```

5.3 Git Sync entre nodos (Engram)

```
# Nodo-1 termina sesion:
engram sync                     # Exporta memorias como chunk comprimido
cd ~/.engram && git add . && git commit -m "sync engram nodo-1" && git

# Nodo-2 inicia sesion:
cd ~/.engram && git pull
engram sync --import            # Importa chunks nuevos de nodo-1
engram search "SPEI"            # Ahora encuentra memorias de nodo-1!
```

Automatizar con hook de Covacha:

```
# En el Agent Loop, despues de release_task:
def post_release_sync():
    """Sincroniza memorias Engram con el repo compartido."""
    subprocess.run(["engram", "sync"])
    subprocess.run(["git", "-C", ENGRAM_DIR, "add", "."])
    subprocess.run(["git", "-C", ENGRAM_DIR, "commit", "-m", f"sync {no
    subprocess.run(["git", "-C", ENGRAM_DIR, "push"])
```

---

6. Ahorro de Tokens Estimado (Stack Completo)

6.1 Sesion de 30 minutos

Sin stack	Solo RTK	RTK + CM	RTK + CM + Engram	Ahorro
118,000 tokens (CLI)	23,900 (-80%)	5,400 (-95%)	5,400 + 500 mem	**-95%**
+ 200,000 (archivos/web)	200,000	12,000 (-94%)	12,000 + 800 mem	**-94%**
318,000 total	**223,900**	**17,400**	**18,700**	**-94%**

6.2 Sesion de 3 horas (con compactacion)

Metrica	Sin stack	Con stack completo
Duracion efectiva	~30 min (compacta y pierde hilo)	3+ horas (CM restaura)
Tokens CLI consumidos	~350,000	~70,000 (RTK)
Tokens archivos/web	~600,000	~36,000 (CM sandbox)
Compactaciones	3-4 (pierde estado)	3-4 (CM restaura automatico)
Decisiones recordadas	0 (session nueva)	Todas (Engram persiste)
Coordinacion	0 (manual)	Automatica (Covacha)

---

7. Covacha MCP + Engram: Bridge de Learnings

7.1 Problema

Engram es local (SQLite por nodo). Covacha es remoto (DynamoDB compartido). Cuando nodo-1 aprende algo, nodo-2 no lo sabe via Engram (diferente SQLite). Pero nodo-2 SI lo ve via Covacha LEARNING#.

7.2 Solucion: Dual-Write + Engram Git Sync

```
Al guardar un learning:

1. Engram (local, formato rico):
   mem_save(title, type, content con What/Why/Where/Learned, topic_key)
   → Busqueda full-text, timeline, progressive disclosure

2. Covacha (remoto, formato compacto):
   save_learning(module, learning_one_liner)
   → Disponible INMEDIATAMENTE para otros nodos via bootstrap

3. Engram Git Sync (async, formato rico):
   engram sync → git push
   → Otros nodos importan en su proximo git pull
   → Busqueda full-text cross-nodo (eventual consistency)
```

7.3 Cual consultar y cuando

```
Misma sesion, mismo nodo:
  → Context Mode (ya esta en el contexto)

Otra sesion, mismo nodo:
  → Engram mem_search (SQLite local, rapido)

Otro nodo, necesidad inmediata:
  → Covacha get_learnings (DynamoDB, real-time)

Otro nodo, investigacion profunda:
  → Engram mem_search (despues de git sync, formato rico)
```

---

8. Node Config Actualizado (.covacha-node.yml)

```yaml
# ~/.covacha-node.yml
node_id: "cesar-macbook-pro"

# Tenant
tenant:
  id: "baatdigital"
  github_org: "baatdigital"

# Workspaces
workspaces:
  - id: "superpago"
    repos:
      - path: /Users/casp/sandboxes/superpago/covacha-payment
        name: covacha-payment
      - path: /Users/casp/sandboxes/superpago/covacha-core
        name: covacha-core
    capabilities: [backend, testing]
    roles: [developer, tech_lead]
  - id: "marketing"
    repos:
      - path: /Users/casp/sandboxes/superpago/mf-marketing
        name: mf-marketing
    capabilities: [frontend]
    roles: [developer]

# Stack (las 4 capas)
stack:
  rtk:
    enabled: true
    version_min: "0.28.0"
  context_mode:
    enabled: true
    plugin: true          # Instalado como plugin de Claude Code
  engram:
    enabled: true
    plugin: true          # Instalado como plugin de Claude Code
    git_sync:
      enabled: true
      repo: "git@github.com:baatdigital/engram-memories.git"
      auto_sync_on_release: true
  covacha:
    enabled: true

# Preferences
preferred_model: "sonnet"
max_concurrent_tasks: 1
auto_switch_workspace: true
```

---

9. Comparativa Final

Pregunta	Herramienta	Por que
"git status es muy largo"	RTK	Comprime CLI output 80%
"git log de 500 commits"	Context Mode	Sandbox, solo relevante entra
"Perdi el hilo despues de compactar"	Context Mode	Restaura archivos, tareas, decisiones
"Que decidimos sobre auth la semana pasada?"	Engram	mem_search, persistente cross-sesion
"Que patron usamos para SPEI?"	Engram	topic_key: "pattern/spei-*"
"Quien esta trabajando en que?"	Covacha	swarm_status, NODE# registry
"ISS-044 depende de ISS-043?"	Covacha	DEP# dependency manager
"Cual es el contrato del endpoint?"	Covacha	CONTRACT# registry
"ISS-043 paso QA?"	Covacha	evaluate_ticket + MSG flow
"Que aprendio nodo-1 sobre el bug?"	Covacha (inmediato) + Engram (profundo)	Dual-write

---

10. Plan de Implementacion Actualizado

Fase 0: Instalar RTK + Context Mode + Engram (1 hora)

```
# En cada nodo:
brew install rtk
rtk init --global

# Context Mode
/plugin marketplace add mksglu/context-mode
/plugin install context-mode@context-mode

# Engram
brew install gentleman-programming/tap/engram
claude plugin marketplace add Gentleman-Programming/engram
claude plugin install engram

# Crear repo compartido de memorias
gh repo create baatdigital/engram-memories --private
```

Fase 1: Covacha MCP Foundation (ya implementado parcialmente)

Lo que ya hicimos hoy:

- [x] node_config.py (auto-detect repos, capabilities, .covacha-node.yml)
- [x] node_registry.py (register, heartbeat, discover, stale detection)
- [x] dependency_manager.py (add, resolve, cycle detection)
- [x] message_bus.py (send, read, mark_as_read, broadcast/direct)

- [x] contract_registry.py (publish, get, search, consumers)
- [x] config.py updates (heartbeat, stale, node TTL, roles)
- [x] 51 tests nuevos, todos pasan

Pendiente:

- [ ] Actualizar CLIs existentes (claim_task, release_task, bootstrap) para usar NODE#
- [ ] MCP Server TypeScript wrapping los modulos Python
- [ ] Integrar Engram dual-write en release_task

Fase 2: MCP Server TypeScript (2-3 dias)

- [ ] Setup proyecto TypeScript con @modelcontextprotocol/sdk
- [ ] 14 tools wrapping los modulos Python via subprocess
- [ ] Publicar como npm package (covacha-mcp-server)
- [ ] Tests con vitest

Fase 3: Agent Loop + Engram Bridge (3-5 dias)

- [ ] agent_loop.py (daemon autonomo)
- [ ] Engram dual-write (mem_save + save_learning)
- [ ] Engram git sync automatico post-release
- [ ] prompt_generator.py con contexto de Engram + Covacha

Fase 4: Evaluacion + Roles (2-3 dias)

- [ ] evaluate_ticket (4 tipos: pre_release, qa, po, post_merge)
- [ ] Role-based message processing
- [ ] Dispatcher inteligente

Fase 5: Multi-Tenant (2-3 dias)

- [ ] Tenant/workspace CRUD
- [ ] Migracion de datos existentes
- [ ] CLI covacha tenant/workspace commands